Creating a Java Stored Procedure in Oracle Database

This blog post provides a quick (and dirty) way of creating Java Stored Procedures in Oracle Database because I can't ever remember how to do that. The Java Developer's Guide details the use of Java in the database, chapter 5 explains how to create Java Stored Procedures. Please refer to the documentation for a *proper* discussion of Java in the Oracle database.

This blog was written using

- Oracle 21c Enterprise Edition patched to 21.7.0
- Oracle Linux 8.6
- VirtualBox 6.1.40

Java stored procedures are written in Java (unsurprisingly). Before they can be used they have to be made available to the PL/SQL and SQL layer of the Oracle database. Therefore there are a few extra steps involved compared to writing stored procedures in PL/SQL.

Creating the Java Source

Rather than relying on the `loadjava` tool this post uses the CREATE JAVA command to create and compile the Java source. Note that errors in the code are not reported back to you so make sure that what you're loading into the database is valid Java and complies with the requirements for Java Stored Procedures (like using static functions etc).

```
CREATE OR REPLACE AND RESOLVE JAVA SOURCE NAMED helloClassSRC AS
public class HelloClass {
    public static String hello( String who )  {
        return "hello, " + who ;
    }
}
/
```

This creates 2 new objects in the schema, a JAVA source and its associated class in my current schema.

```
SELECT
    object_name,
    object_type
FROM
    user_objects
WHERE
    created > sysdate - 1;

      OBJECT_NAME    OBJECT_TYPE
_________________ _______________
HelloClass        JAVA CLASS
HELLOCLASS_SRC    JAVA SOURCE
```

With the Java class stored in the database the next step is to make it available to the SQL and PL/SQL layers.

Publishing the Java Class

The `hello()` Java function returns a string, and I'm going to do the same with the PL/SQL call specification.

```
CREATE FUNCTION hello_java (
    p_who VARCHAR2
) RETURN VARCHAR2
AS LANGUAGE JAVA
NAME 'HelloClass.hello(java.lang.String) return java.lang.String';
/
```

The `hello_java` (PL/SQL!) function takes a single argument, `p_who` of (database) type `VARCHAR2` and returns a `VARCHAR2`. The function is then mapped to the static `hello()` function in `HelloClass`, which is where you enter the Java world. `hello()` takes a string as an input parameter and returns a string.

Using hello_java

Once the PL/SQL call specification is created, it's very easy to use the function:

```
SELECT
    hello_java('world') AS greeting
FROM
    dual;

       GREETING
```

```
hello, world
```

Although I named the function `hello_java`, there is no need to specify that Java is used under the covers. It just makes it easier for me to see that this function isn't a PL/SQL but rather a Java function. Any valid PL/SQL identifier can be used in the call specification. Speaking of PL/SQL, I can of course use `hello_java()` in PL/SQL:

```
DECLARE
    l_string VARCHAR2(100);
BEGIN
    l_string := hello_java('world');
    DBMS_OUTPUT.PUT_LINE(l_string);
END;
/
```

Which prints "hello, world" once serveroutput is enabled.

Summary

I can never remember how to create Java stored procedures and hope this post helps you save 5 minutes as it does for me. There is of course a lot more to say about the topic, so please head over to the Java Developer's Guide for more details.